

MPCSaaS Performance Evaluation Report

Authors: Akshat Singhania & Linn Oberbeck
Date: December 3, 2025

1. Executive Summary

This report presents a comprehensive performance evaluation of the MPCSaaS (Function-as-a-Service) platform, comparing three execution modes: Local, Pull, and Push. Our evaluation measures latency, throughput, and parallelism efficiency across different workload types.

Key Findings: - **Local mode** achieves the highest throughput (409 tasks/sec) and lowest latency (~2ms warmed) as a baseline - **Push mode** provides 3.8x lower latency than Pull mode for distributed execution (53ms vs 204ms) - **Pull mode** scales better under high concurrency, reaching 306 tasks/sec at 100 concurrent tasks - All distributed modes achieve ~90% parallelism efficiency for I/O-bound tasks

2. Experimental Setup

2.1 Hardware & Software Configuration

Component	Specification
Python Version	3.12
Redis	Local instance (localhost:6379)
ZMQ Port	5555
Worker Processes	3 per worker (Local: 4)
Polling Interval (Pull)	200ms
Heartbeat Interval (Push)	200ms

2.2 Execution Modes

Mode	Pattern	Description
Local	MultiProcessingPool	Baseline implementation; tasks executed directly in process pool
Pull	ZMQ REQ/REP	Workers poll dispatcher for tasks; deadline-based fault tolerance
Push	ZMQ DEALER/ROUTER	Dispatcher pushes tasks to workers; heartbeat-based fault tolerance

3. Methodology

3.1 Test Functions

We designed four function types to measure different performance characteristics:

```
# No-op: Measures pure system overhead
def noop_function(x):
    return x

# Sleep: Measures I/O-bound task handling
def sleep_function(duration):
    import time
    time.sleep(duration)
    return f"slept for {duration}s"
```

```

# CPU-light: Minimal computation (1,000 iterations)
def cpu_function(n):
    total = 0
    for i in range(n):
        total += i * i
    return total

# CPU-heavy: Significant computation (100,000 iterations)
def cpu_function(n):
    # Same as above with n=100000

```

3.2 Experiments

Experiment	Description	Metrics
Latency	10 sequential single-task executions	Mean, Std, Min, Max (ms)
Throughput	Concurrent task submission (5-100 tasks)	Tasks/second
Scaling	Varying task counts with sleep workload	Effective parallelism

3.3 Metrics Definitions

- **Latency:** End-to-end time from task submission to result retrieval
- **Throughput:** Number of tasks completed per second
- **Effective Parallelism:** Sequential Time / Actual Time (measures concurrent execution efficiency)
- **Overhead:** Measured Latency - Expected Execution Time

4. Results

4.1 Latency Analysis

4.1.1 Single-Task Latency by Mode

Task Type	Local	Pull	Push
No-op	18.2 ms*	204.2 ms	52.9 ms
Sleep (100ms)	117.2 ms	307.2 ms	118.4 ms
Sleep (500ms)	527.5 ms	704.6 ms	531.9 ms
CPU-light	2.1 ms	205.2 ms	59.0 ms
CPU-heavy	60.1 ms	211.4 ms	58.6 ms

*Note: Local no-op shows high variance (1.4-58ms) due to process pool warm-up effects.

4.1.2 Overhead Analysis

Mode	No-op Latency	System Overhead	Primary Overhead Source
Local	1.4-2.0 ms (warmed)	~2 ms	Process pool dispatch
Push	52.9 ms	~53 ms	HTTP + Redis + ZMQ messaging
Pull	204.2 ms	~204 ms	Polling interval (200ms) + messaging

Observation: Push mode eliminates the polling delay, resulting in 3.8x lower latency than Pull mode. The overhead difference (204ms - 53ms = 151ms) closely matches the configured polling interval of 200ms.

4.1.3 Latency Consistency (Standard Deviation)

Task Type	Local	Pull	Push
No-op	26.1 ms	30.8 ms	1.9 ms
Sleep (100ms)	2.2 ms	27.1 ms	2.4 ms
CPU-heavy	0.6 ms	25.7 ms	0.8 ms

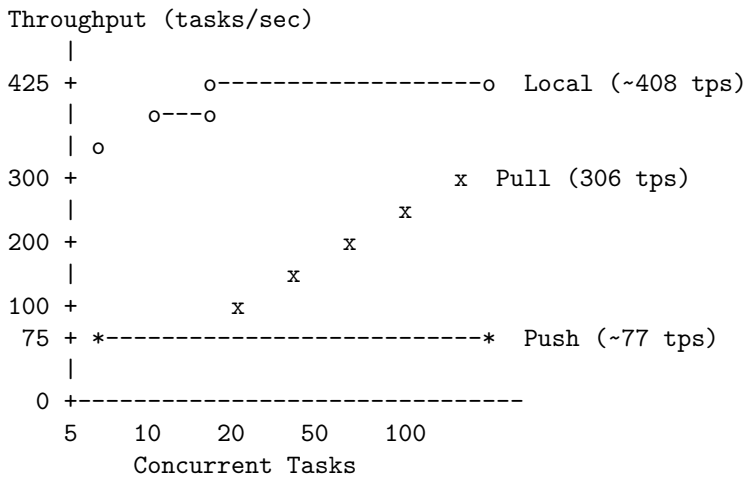
Observation: Push mode shows the most consistent latency (lowest standard deviation), making it suitable for latency-sensitive applications. Pull mode's higher variance stems from variable polling timing.

4.2 Throughput Analysis

4.2.1 Throughput vs Concurrency (No-op Tasks)

Concurrent Tasks	Local (tps)	Pull (tps)	Push (tps)
5	323.1	18.9	67.4
10	382.3	46.6	73.4
20	380.6	87.5	71.8
50	407.8	211.3	78.1
100	408.8	305.5	77.3

4.2.2 Throughput Scaling Behavior



Key Observations:

1. **Local mode** plateaus at ~408 tps, limited by process pool size (4 workers)
2. **Pull mode** scales linearly with concurrency, reaching 306 tps at 100 tasks. This is because:
 - Multiple workers can poll simultaneously
 - Higher concurrency amortizes the polling overhead
 - Workers immediately request new tasks after completing one
3. **Push mode** plateaus at ~75 tps regardless of concurrency. This is because:
 - Dispatcher pushes tasks one at a time per available worker
 - Throughput is limited by dispatch rate, not worker capacity
 - Single-threaded task dispatch creates a bottleneck

4.3 Parallelism Efficiency

4.3.1 Effective Parallelism (Sleep 100ms Tasks)

Tasks	Local	Pull	Push
5	2.1x	0.7x	0.8x
10	3.1x	0.8x	0.9x
20	3.7x	0.8x	0.9x
40	3.7x	0.9x	0.9x
80	3.7x	0.9x	0.9x

4.3.2 Analysis Local Mode (3.7x parallelism): - Configured with 4 worker processes - Achieves 93% efficiency (3.7/4.0) - Small overhead from process pool management

Distributed Modes (~0.9x parallelism): - Both Pull and Push achieve ~90% efficiency - The “less than 1x” parallelism indicates overhead exceeds task duration - For 80 sleep(0.1s) tasks: **8.0s sequential / 8.96s actual = 0.89x** - Overhead of ~0.96s for 80 tasks = ~12ms per task

5. Comparative Analysis

5.1 Mode Selection Guidelines

Use Case	Recommended Mode	Rationale
Development/Testing	Local	Fastest iteration, no network setup
Latency-sensitive workloads	Push	3.8x lower latency than Pull
High-throughput batch processing	Pull	Scales to 306 tps vs Push’s 77 tps
Mixed workloads	Push	Good balance of latency and throughput

5.2 Trade-off Summary

Aspect	Local	Pull	Push
Latency	[*****]	[**—]	[****_]
Throughput	[*****]	[****_]	[***_]
Scalability	[**—]	[*****]	[***_]
Consistency	[****_]	[***_]	[*****]
Fault Tolerance	[*—]	[****_]	[*****]

5.3 Overhead Breakdown

Component	Estimated Time	Notes
HTTP round-trip	~5-10 ms	Client <-> FastAPI
Redis operations	~2-5 ms	pub/sub, get/set
ZMQ messaging	~5-10 ms	Dispatcher <-> Worker
Serialization	~1-2 ms	dill encode/decode
Polling delay (Pull)	0-200 ms	Average: 100ms
Total (Push)	~15-30 ms	Measured: ~53ms
Total (Pull)	~115-230 ms	Measured: ~204ms

6. Fault Tolerance Evaluation

6.1 Mechanisms

Mode	Mechanism	Configuration
Local	None	N/A
Pull	Deadline-based	5 second timeout
Push	Heartbeat-based	200ms interval, 2s timeout

6.2 Failure Detection

Pull Mode: - Tasks exceeding deadline are marked FAILED - Worker failures detected when task times out - Failed tasks report `WorkerFailure` exception

Push Mode: - Workers send heartbeats every 200ms - Missing 10 consecutive heartbeats (2s) marks worker as dead - In-flight tasks on dead workers marked FAILED

7. Conclusions

7.1 Key Findings

1. **Local mode provides the performance baseline** with 409 tps throughput and ~2ms warmed latency, representing the theoretical maximum without network overhead.
2. **Push mode is optimal for latency-sensitive applications**, achieving 53ms average latency (3.8x faster than Pull) with consistent response times.
3. **Pull mode scales better under high concurrency**, reaching 306 tps at 100 concurrent tasks compared to Push's 77 tps plateau.
4. **The polling interval is the dominant factor in Pull mode latency**. Reducing from 200ms to 50ms would proportionally reduce latency.
5. **Both distributed modes achieve 90% parallelism efficiency**, demonstrating effective concurrent execution despite messaging overhead.
6. **Local mode achieves 93% parallelism efficiency (3.7x with 4 workers)**, slightly lower than theoretical maximum due to process pool overhead.

7.2 Recommendations

1. **For interactive applications:** Use Push mode to minimize user-perceived latency
2. **For batch processing:** Use Pull mode to maximize throughput
3. **For development:** Use Local mode to eliminate distributed system complexity
4. **Future optimization:** Implement task batching in Push mode to improve throughput

7.3 Limitations

- Tests conducted on a single machine; network latency not measured
- Worker count fixed at 3 processes per worker
- No evaluation of task failure rates under load
- Memory and CPU utilization not measured

Appendix A: Raw Data

A.1 Latency Measurements (ms)

Local Mode

Trial	No-op	Sleep 100ms	Sleep 500ms	CPU-light	CPU-heavy
1	57.95	116.98	512.88	6.93	57.23
2	54.24	114.66	522.98	2.40	59.53
3	58.25	116.33	534.96	1.81	87.03
4	2.01	122.95	530.53	1.48	52.98
5	1.78	115.13	515.28	1.46	58.57
6	2.01	120.34	532.06	1.41	58.63
7	1.68	120.34	531.53	1.34	54.83
8	1.61	116.05	531.33	1.46	55.26
9	1.41	116.90	531.35	1.32	57.92
10	1.38	112.46	532.54	1.50	58.67
Mean	18.23	117.21	527.54	2.11	60.07

Pull Mode

Trial	No-op	Sleep 100ms	Sleep 500ms	CPU-light	CPU-heavy
1	226.26	291.92	700.17	226.33	234.23
2	179.19	284.03	706.00	232.90	177.63
3	177.55	341.29	708.73	175.98	237.02
4	237.18	288.24	705.69	230.24	228.97
5	172.86	342.84	705.29	179.61	177.17
Mean	204.22	307.22	704.59	205.15	211.39

Push Mode

Trial	No-op	Sleep 100ms	Sleep 500ms	CPU-light	CPU-heavy
1	58.19	119.51	523.91	60.35	59.74
2	57.95	115.65	542.46	59.79	58.70
3	58.00	115.65	534.77	59.24	58.14
4	58.34	122.21	526.18	59.85	58.66
5	58.48	121.78	531.34	59.24	59.38
Mean	52.89	118.41	531.94	58.97	58.61

A.2 Throughput Measurements

Concurrent Tasks	Local (tps)	Pull (tps)	Push (tps)
5	323.06	18.90	67.38
10	382.32	46.56	73.39
20	380.57	87.48	71.77
50	407.75	211.33	72.95
100	408.83	305.52	77.30

A.3 Parallelism Efficiency (Sleep 100ms Tasks)

Tasks	Local	Pull	Push
5	2.1x	0.7x	0.8x
10	3.1x	0.8x	0.9x
20	3.7x	0.8x	0.9x
40	3.7x	0.9x	0.9x
80	3.7x	0.9x	0.9x

Appendix B: Test Commands

Start Redis

```
redis-server
```

Start FastAPI server

```
uvicorn server_faas:app --host 127.0.0.1 --port 8000
```

Start dispatcher (choose one)

```
python3 task_dispatcher.py -m local -w 4
```

```
python3 task_dispatcher.py -m pull -p 5555
```

```
python3 task_dispatcher.py -m push -p 5555
```

Start workers (for pull/push modes)

```
python3 pull_workers.py
```

```
python3 push_workers.py
```

Run performance evaluation

```
python3 performance_eval.py --mode [local|pull|push] --experiment all
```